

EduGenie Project Documentation Template

https://gemini.google.com/app/6e3a2f0f88a5b6b0

User prompt: AI-ML-and-GEN-AI-Track-Project-Template Repository Structure 1.Brainstorming & Ideation 2.Requirement Analysis 3.Project Design Phase 4.Project Planning Phase 5.Project Development Phase 6.Project Testing 7.Project Documentation 8.Project Demonstration Replace the placeholder files with your team's project deliverables. Etha yallarhaiyum document ta create pannu tha

Response: EduGenie: Google Gemini Powered Learning Assistant

1. Brainstorming & Ideation

- **Project Concept:** EduGenie is a lightweight AI-powered educational assistant that simplifies complex learning materials and provides personalized study assistance.
- **Problem Statement:** Traditional educational content can be overly complex or static, making self-directed learning difficult for students lacking tailored guidance.
- **Target Audience:** Designed for students across all academic levels, self-learners, and educators requiring accessible learning tools.
- **Core Value Proposition:** Operates efficiently on accessible consumer hardware (such as Apple Silicon Mac M1) by combining cloud-based generative AI with lightweight local language models.
- **Key Functional Capabilities Identified:**
 - **Question & Answer:** Smart, direct answers to academic and general knowledge questions.
 - **Concept Explanation:** Simplified, beginner-friendly breakdowns of complex concepts.
 - **Interactive Quiz Generation:** Automated multiple-choice question generation with instant grading.
 - **Text Summarization:** Concise summaries of lengthy educational passages.
 - **Structured Learning Paths:** Step-by-step topic roadmaps categorized by difficulty level.

2. Requirement Analysis

Functional Requirements

- **/qa Endpoint:** Accepts user queries via HTTP GET requests and returns concise factual answers.
- **/explain/ Endpoint:** Accepts topic names via HTTP POST requests and generates simplified explanations suitable for school students.
- **/quiz Endpoint:** Accepts text passages via HTTP POST requests and produces 3 multiple-choice questions in structured JSON format.
- **/summarize/ Endpoint:** Accepts long text blocks via HTTP POST requests and generates concise summaries retaining key information.
- **/learn/recommendations Endpoint:** Accepts subject names via HTTP GET requests and provides structured learning roadmaps with recommended study resources.

Non-Functional Requirements

- **Performance & Lightweight Footprint:** Designed to run efficiently on standard developer devices (e.g., Mac M1) without heavy GPU hardware dependencies.
- **Modularity:** Individual AI tasks are decoupled into separate Python source files for ease of maintenance and upgrades.
- **Error Resilience:** Includes fallback mechanisms for API timeouts, response extraction issues, and JSON parsing cleanups.

Technical Dependencies

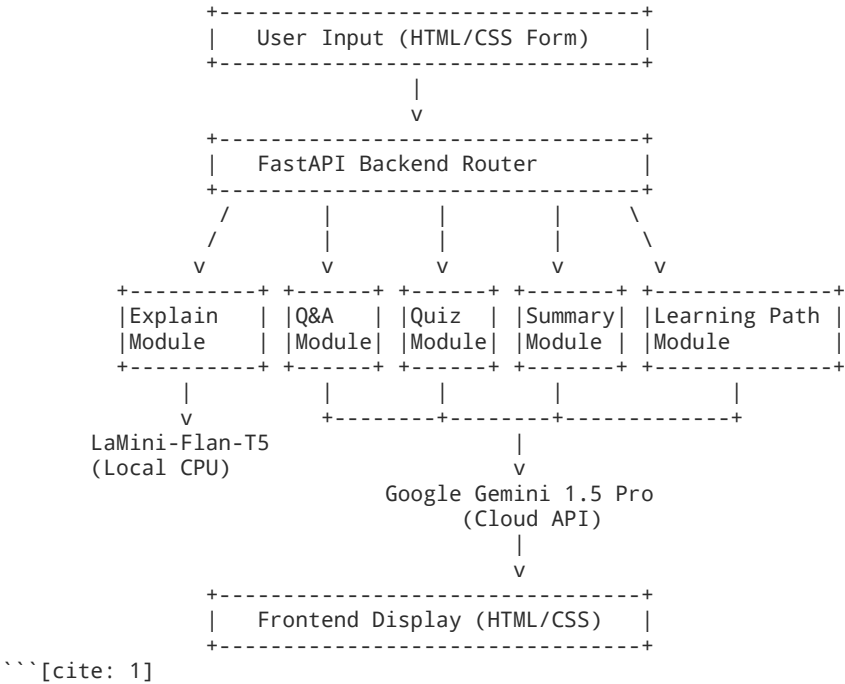
Component	Technology	Role / Function
Language	Python 3.10+	Core programming environment
Backend Framework	FastAPI	High-performance REST API routing and request handling
Server Engine	Uvicorn	Asynchronous Server Gateway Interface (ASGI) server
Templating Engine	Jinja2	Renders dynamic HTML views on the frontend
Cloud AI Model	Google Gemini 1.5 Pro	Handles Q&A, quiz generation, summarization, and learning paths

Component	Technology	Role / Function
Local AI Model	LaMini-Flan-T5-783M	Runs locally on CPU/PyTorch to provide concept explanations

3. Project Design Phase

System Architecture

The application follows a client-server architecture where user inputs from an HTML/CSS frontend are routed through a FastAPI backend to either cloud-based or local AI models.



Repository Folder Structure

EduGenie/ | — main.py # Main FastAPI application and API routes | — explanation_module.py # Concept explanation logic using LaMini-Flan-T5
| — qna.py # Question answering logic using Gemini 1.5 Pro | — quiz_module.py # Quiz generation and JSON parsing logic | —
summary_module.py # Text summarization logic using Gemini 1.5 Pro | — learning_path.py # Structured learning roadmap generation | —
requirements.txt # Project dependencies | | — templates/ | | — index.html # HTML5 User Interface | | — static/ | — style.css # Frontend
styling and layout

4. Project Planning Phase

Milestone Breakdown
* **Milestone 1: Model Selection & Architecture**
* **Milestone 2: Core Functionalities Development**
* **Milestone 3: Frontend Development & Integration**
* **Milestone 4: Deployment & Functional Testing**

Target User Scenarios
* **Scenario 1 (General Knowledge Q&A)**
* **Scenario 2 (Assessment & Practice)**
* **Scenario 3 (Structured Learning)**

5. Project Development Phase

Module Implementation

1. Concept Explanation Module (`explanation\_module.py`)
Uses the local `MBZUAI/LaMini-Flan-T5-783M` model via Hugging Face Transformers for offline, CPU-friendly conc

```

```python
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
 input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
 inputs = explain_tokenizer(input_text, return_tensors="pt")

 outputs = explain_model.generate(
 **inputs,
 max_new_tokens=150,
 temperature=0.7,
 top_k=50,
 top_p=0.95,
 do_sample=True
)

 explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
 return explanation
```[cite: 1]

```

2. Question Answering Module (`qna.py`)
Utilizes Google Gemini 1.5 Pro to process open-ended academic queries[cite: 1].

```

```python
import google.generativeai as genai

def answer_question_with_gemini(question: str) -> str:
 try:
 model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
 response = model.generate_content(question)
 return response.text.strip()
 except Exception as e:
 return f"⚠️ Error in QnA: {e}"
```[cite: 1]

```

3. Quiz Generation Module (`quiz\_module.py`)
Generates 3 MCQs in JSON format from a given text passage and strips markdown code fences[cite: 1].

```

```python
import re
import google.generativeai as genai

def clean_json_block(text: str):
 return re.sub(r"```(?:json)?\n(?:\n```", r"\1", text, flags=re.DOTALL).strip()

```

```

def generate_quiz(text: str) -> list:
 try:

```

```

 model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
 prompt = f"""

```

You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:

- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:

```

[
 {
 "question": "What is ...?",
 "options": ["A", "B", "C", "D"],
 "answer": "A"
 }
]

```

Passage:

```
{text}
```

```
"""
```

```

 response = model.generate_content(prompt)
 quiz_text = response.text.strip()
 cleaned_text = clean_json_block(quiz_text)

```

```

 return cleaned_text
 except Exception as e:
 return f"⚠️ Error in Quiz Generation: {e}"
```[cite: 1]

```

4. Text Summarization Module (`summary\_module.py`)
Condenses long educational texts while maintaining core context[cite: 1].

```

```python
import google.generativeai as genai

def summarize_text(text: str) -> str:
 try:
 model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
 prompt = f"Summarize the following text in simple language:\n\n{text}"
 response = model.generate_content(prompt)
 return response.text.strip()
 except Exception as e:
 return f"⚠️ Error in Summary: {e}"
```[cite: 1]

```

5. Learning Path Module (`learning\_path.py`)
Generates multi-stage learning roadmaps complete with topic rankings and study resource links[cite: 1].

```

```python
import traceback
import google.generativeai as genai

def get_learning_recommendations(topic: str):
 prompt = f"""
You are an AI tutor. The student wants to learn about: {topic}.
Suggest a structured and adaptive learning path including key topics, order of learning, and resources.
Include beginner, intermediate, and advanced levels if needed.
"""
 try:
 model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
 response = model.generate_content(prompt)

 if hasattr(response, "text"):
 return response.text
 elif hasattr(response, "parts") and response.parts:
 return response.parts[0].text
 else:
 return "❌ Could not extract content from Gemini response."
 except Exception as e:
 traceback.print_exc()
 return f"❌ Error occurred: {str(e)}"
```[cite: 1]

```

6. Application Server (`main.py`)
Exposes REST endpoints and links them to module handlers[cite: 1].

```

```python
from fastapi import FastAPI, Request, Query
from fastapi.responses import JSONResponse
from qna import answer_question_with_gemini
from explanation_module import explain_topic
from summary_module import summarize_text
from quiz_module import generate_quiz
from learning_path import get_learning_recommendations

app = FastAPI()

@app.get("/qa")
async def answer_question(question: str = Query(...)):
 answer = answer_question_with_gemini(question)
 return {"answer": answer}

@app.post("/explain/")
async def explain_api(request: Request):
 data = await request.json()
 topic = data.get("topic")
 if not topic:
 return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)

```

```

 explanation = explain_topic(topic)
 return {"topic": topic, "explanation": explanation}

@app.post("/summarize/")
async def summarize_api(request: Request):
 data = await request.json()
 text = data.get("text")
 if not text:
 return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
 summary = summarize_text(text)
 return {"summary": summary}

@app.post("/quiz")
async def quiz_api(request: Request):
 data = await request.json()
 text = data.get("text")
 if not text:
 return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
 quiz = generate_quiz(text)
 return JSONResponse(content={"quiz": quiz})

@app.get("/learn/recommendations")
async def learning_recommendation_api(topic: str = Query(...)):
 recommendation = get_learning_recommendations(topic)
 return {"topic": topic, "recommendation": recommendation}
` `[cite: 1]

```

---

## ## 6. Project Testing

### ### Test Suite Execution & Output Verification

Feature Module	Input / Test Case	Expected Result	Verification Status
**Q&A Module**	`"which is the largest ocean?"`	Returns `"The Pacific Ocean is the largest ocean on Earth"	Pass
**Explanation Module**	`"quantum computing"`	Produces an intuitive breakdown of qubits and quantum mecha	Pass
**Explanation Module**	`"Binary Search Algorithm"`	Explains index lookup in a sorted list using the need	Pass
**Summarization Module**	Historical text on Industrial Revolution	Produces a simplified summary covering	Pass
**Quiz Module**	Topic: `"Pythagoras theorem"`	Generates 3 MCQs with 4 options each; validates correct an	Pass
**Learning Path Module**	Topic: `"SQL"`	Returns 3-tier roadmap (Beginner, Intermediate, Advanced) with t	Pass

---

## ## 7. Project Documentation

### ### Installation & Environment Setup Guide

#### #### Prerequisites

- \* Python 3.10 or higher[cite: 1]
- \* Google Gemini API Key[cite: 1]

#### #### 1. Setup Python Environment

```

` ` `bash
python --version
` `[cite: 1]

```

#### #### 2. Install Required Packages

```

` ` `bash
pip install fastapi uvicorn jinja2 torch transformers google-generativeai
` `[cite: 1]

```

#### #### 3. Configure API Key

```

Set up your Google Gemini API key from Google AI Studio in your environment variables[cite: 1]:
` ` `bash
export GEMINI_API_KEY="your_api_key_here"
` `[cite: 1]

```

#### #### 4. Launch Application

```

Start the Uvicorn ASGI server[cite: 1]:
` ` `bash
uvicorn main:app --reload
` `[cite: 1]

```

Access the web dashboard in your browser at `[http://127.0.0.1:8000](http://127.0.0.1:8000)`[cite: 1].

### ### Project Submission Details

\* **Author**: Tella Divya Sree[cite: 1]

\* **Mentor**: Siri[cite: 1]

\* **Submission Date**: 11/04/2025[cite: 1]

---

## ## 8. Project Demonstration & Future Roadmap

### ### Interface Summary

\* **Integrated Frontend View**: Single-page web dashboard providing tabbed or form-based entry points for Q&A,

\* **Real-time Results**: Responses render dynamically below each form field without page reloads[cite: 1].

### ### Future Development Roadmap

\* **Voice-Based Interaction**: Integrate speech-to-text and text-to-speech engines to enable hands-free intera

\* **Multilingual Support**: Expand language translation capabilities for wider global accessibility[cite: 1].

\* **Mobile Application Development**: Build a cross-platform mobile frontend with offline caching capabilities

\* **Progress Tracking & Gamification**: Add user dashboards with learning metrics, streaks, and challenge badg

\* **LMS Integration & File Scanning**: Support direct upload and optical character recognition (OCR) for scann

---